Installing OpenShift SNO on Hetzner Dedicated Servers with aicli

This guide installs Single Node OpenShift (SNO) on a Hetzner dedicated server using `aicli` — the assisted installer CLI. It handles Hetzner's unusual /32 networking, EFI boot, and dual-NVMe disk layout.

> **Why aicli over the agent-based installer?** The agent-based installer is harder to debug on Hetzner — there's no KVM console, and if networking fails in the live ISO, you're blind. With aicli, the discovery ISO boots, registers with the Red Hat hosted API, and you can inspect/fix host details iteratively before starting the install.

Prerequisites

Hardware

Model	CPU	RAM	Storage	Price
AX42/AX52	AMD Ryzen 7 7700 (8c/16t)	64 GB DDR5	2x 1TB NVMe	~55 EUR/month

Order from Hetzner Robot. Select **Rescue System** as the initial OS.

Your Workstation

```
# Install aicli
pip3 install aicli

# Install oc CLI
wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/stable-4.22/openshift-client-linux.tar.gz
tar xzf openshift-client-linux.tar.gz
sudo mv oc kubectl /usr/local/bin/
```

Accounts and Secrets

- **Red Hat pull secret** — download from console.redhat.com/openshift/install/pull-secret, save as `pull-secret.json`
- **Red Hat offline token** — get from console.redhat.com/openshift/token (needed for aicli to authenticate with the assisted installer API)
- **SSH public key** — `ssh-keygen -t ed25519 -f ~/.ssh/hetzner-sno -N ""` if you don't have one

DNS Records

Create these A records pointing to your server's public IP **before starting**:

Type	Name	Content
A	`api.<cluster>.<domain>`	`<SERVER_IP>`
A	`*.apps.<cluster>.<domain>`	`<SERVER_IP>`

Example for this guide:

```
api.ocp4.lapthorn.me       →  157.180.56.29
*.apps.ocp4.lapthorn.me    →  157.180.56.29
```

Step 1: Gather Server Details

SSH into the Hetzner rescue system and record your server's network details:

```
ssh root@<SERVER_IP>

# Find NIC name and MAC address
ip link show | grep -E 'state UP|link/ether'

# Find IP and prefix length
ip addr show

# Find gateway
ip route | grep default
```

Write down these values — you'll need them throughout:

Detail	Example	Your Value
Public IP	157.180.56.29	
NIC name	enp6s0	
MAC address	9c:6b:00:26:4e:7d	
Gateway	157.180.56.1	
Disk 1 (ISO boot)	/dev/nvme0n1	
Disk 2 (OS install target)	/dev/nvme1n1	

Understanding Hetzner /32 Networking

Hetzner assigns a /32 IP (single address, no subnet). This is unusual — most datacenters give you a /24 or /26. The key implication: **the kernel has no route to the gateway** because the gateway isn't on the same subnet.

You must add an explicit **host route to the gateway** before the default route:

```
routes:
  - destination: 157.180.56.1/32       # Host route TO the gateway
    next-hop-interface: enp6s0
  - destination: 0.0.0.0/0             # Default route VIA the gateway
    next-hop-address: 157.180.56.1
    next-hop-interface: enp6s0
```

Without the first route, the system can't reach the gateway and has no network connectivity.

Step 2: Configure aicli

2.1 Set the Offline Token

```
export AI_OFFLINETOKEN='<your-offline-token-from-console.redhat.com>'
```

Or create `~/.aicli/config.yml`:

```
default:
  token: <your-offline-token>
```

2.2 Verify aicli Works

```
aicli list cluster
```

You should see an empty table (or existing clusters if you've used it before).

Step 3: Create the Cluster and InfraEnv

3.1 Create the Cluster Parameters File

Create `cluster-params.yml`:

```
# cluster-params.yml — SNO on Hetzner
openshift_version: "4.22"
sno: true
base_dns_domain: lapthorn.me
pull_secret: pull-secret.json
ssh_public_key: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIF81+EYvBMyyg9dya8VpwmjiE/
UOL3gBxAPo7mrI4I92
api_vips: [157.180.56.29]
ingress_vips: [157.180.56.29]
machine_networks: ["157.180.56.0/26"]
network_type: OVNKubernetes
```

> **Note:** For SNO, `api_vips` and `ingress_vips` are both the server's public IP. `machine_networks` must match the subnet that contains your IP — check Hetzner Robot or `ip route` output. Hetzner typically uses a /26 even though your IP is /32.

3.2 Create the InfraEnv Parameters File

The infraenv contains the static network config that gets baked into the discovery ISO.

Create `infraenv-params.yml`:

```
# infraenv-params.yml — static networking for Hetzner /32
cluster: ocp4
static_network_config:
  - mac_interface_map:
      - logical_nic_name: enp6s0
        mac_address: "9c:6b:00:26:4e:7d"
    network_yaml: |
      interfaces:
        - name: enp6s0
          type: ethernet
          state: up
          ipv4:
            enabled: true
            address:
              - ip: 157.180.56.29
                prefix-length: 32
            dhcp: false
          ipv6:
            enabled: false
      dns-resolver:
        config:
          server:
            - 185.12.64.1
            - 185.12.64.2
      routes:
        config:
          - destination: 157.180.56.1/32
            next-hop-interface: enp6s0
          - destination: 0.0.0.0/0
            next-hop-address: 157.180.56.1
            next-hop-interface: enp6s0
```

> **Critical:** The `prefix-length: 32` and the host route to the gateway (`destination: <gateway>/32`) are essential for Hetzner. Without them, the discovery ISO boots but has no network — and you can't debug it without a KVM console.

3.3 Create the Cluster and InfraEnv

```
aicli create cluster ocp4 --pf cluster-params.yml
aicli create infraenv ocp4 --pf infraenv-params.yml
```

3.4 Download the Discovery ISO

```
aicli download iso ocp4
ls -lh ocp4.iso
```

This produces `ocp4.iso` (~1.2 GB).

Step 4: Boot the Discovery ISO on Hetzner

4.1 Prepare the Server

SSH into the rescue system:

```
ssh root@<SERVER_IP>
```

Wipe both disks:

```
wipefs -af /dev/nvme0n1
wipefs -af /dev/nvme1n1
```

4.2 Upload and Write the ISO

From your workstation, upload the ISO:

```
scp ocp4.iso root@<SERVER_IP>:/root/
```

On the server, write the ISO to **only one disk** (nvme0n1):

```
dd if=/root/ocp4.iso of=/dev/nvme0n1 bs=4M status=progress
sync
```

> **Critical: Only write the ISO to ONE NVMe.** If both disks have the ISO, the live OS auto-mounts the second disk's ISO partition at `/run/media/iso`, which blocks the installer from getting exclusive access to that disk.

4.3 Create the EFI Boot Entry

The server needs to know how to boot from the NVMe. Create an EFI boot entry:

```
# Find the EFI partition on the ISO disk
fdisk -l /dev/nvme0n1 | grep EFI

# Create the boot entry (partition 2 is typical for the EFI System Partition)
efibootmgr -c -d /dev/nvme0n1 -p 2 -l \\EFI\\BOOT\\BOOTX64.EFI -L "OpenShift Discovery"
```

> **Critical:** Use soft `reboot` from the rescue system, NOT the Hetzner Robot "hardware reset" button. A hardware reset can wipe EFI variables on some Hetzner servers.

4.4 Disable Rescue and Reboot

In Hetzner Robot: 1. Go to your server → **Rescue** tab → **Deactivate rescue system** 2. Back on the server SSH session:

```
reboot
```

The server will boot from the discovery ISO. Wait 5-10 minutes for it to come up.

Step 5: Wait for Host Discovery

5.1 Check Host Registration

The discovery ISO boots, configures networking with your static config, and registers with the assisted installer API. Monitor from your workstation:

```
# Check if the host has registered
aicli list host

# Wait for status to become "known"
watch -n 10 'aicli list host'
```

Expected progression:

```
discovering → insufficient → known
```

- **discovering** — host booted, agent is gathering hardware info
- **insufficient** — validations not yet passing (give it a few minutes)
- **known** — ready to install

If the host never appears, the static network config is wrong. Go back to rescue mode and check your NIC name, MAC, IP, and routes.

5.2 Verify Host Details

```
aicli info host <hostname>
```

Check that: - IP address matches your server - NIC name and MAC are correct - Both NVMe disks are detected

5.3 Set the Installation Disk

This is critical. The ISO is on nvme0n1 — you must install the OS to the OTHER disk:

```
# List available disks
aicli info host <hostname> | grep -i disk

# Set the install target to the clean disk
aicli update host <hostname> -P installation_disk_path=/dev/nvme1n1
```

> **If you skip this:** The installer tries to install to the same disk the ISO is on, which fails.

Step 6: Start the Installation

6.1 Verify Readiness

```
aicli info cluster ocp4 | grep -E 'status|validations'
```

The cluster status should be `ready`. If it shows `pending-for-input` or `insufficient`, check the validations:

```
aicli info cluster ocp4 -f validations_info
```

6.2 Start the Install

```
aicli start cluster ocp4
```

6.3 Monitor Progress

```
# Watch overall status
watch -n 30 'aicli info cluster ocp4 | grep -E "status|progress"'

# Watch host progress
watch -n 30 'aicli list host'
```

Expected progression:

```
preparing-for-installation → installing → installing-in-progress → installed
```

The install takes **35-60 minutes**. Key phases: 1. **Writing to disk** (~5 min) — installer writes RHCOS to nvme1n1 2. **Rebooting** (~2 min) — server reboots from nvme1n1 3. **Configuring** (~10 min) — first boot, ignition applies 4. **Bootstrapping** (~15-25 min) — control plane operators start 5. **Finalizing** (~5-10 min) — cluster operators reach Available

6.4 Handle the Post-Install Reboot

After the installer writes to disk, the server reboots. It needs to boot from **nvme1n1** (the install target), not nvme0n1 (the ISO).

If the server becomes unreachable after reboot: 1. Go to Hetzner Robot → **Rescue** tab → **Activate rescue system** 2. **Reset** the server (hardware reset is OK here — the install is already on disk) 3. SSH into rescue 4. Create an EFI boot entry for the installed OS:

```
# Mount the EFI partition of the install disk
mkdir -p /mnt/efi
mount /dev/nvme1n1p2 /mnt/efi
ls /mnt/efi/EFI/

# Create boot entry
efibootmgr -c -d /dev/nvme1n1 -p 2 -l \\EFI\\redhat\\shimx64.efi -L "Red Hat Enterprise
Linux"

# Set it as first boot option
efibootmgr    # Note the boot number (e.g., 0005)
efibootmgr -o 0005,<other-entries>

# Deactivate rescue in Robot, then soft reboot
reboot
```

Step 7: Download Credentials

Once the install completes:

```
# Download kubeconfig
aicli download kubeconfig ocp4
# Saves to ocp4/kubeconfig

# Get kubeadmin password
aicli info cluster ocp4 | grep -i password
# Or download the full credentials
aicli download credentials ocp4
```

Move the kubeconfig:

```
mkdir -p ~/sno-install
cp ocp4/kubeconfig ~/sno-install/kubeconfig.ocp4
export KUBECONFIG=~/sno-install/kubeconfig.ocp4
```

Step 8: Verify the Cluster

```
export KUBECONFIG=~/sno-install/kubeconfig.ocp4

# Cluster version
oc get clusterversion

# Node status
oc get nodes

# All cluster operators should be Available
oc get co

# Web console URL
oc whoami --show-console
```

Expected output:

```
NAME      VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE
version   4.22.14   True        False         False      5m

NAME                                        STATUS   ROLES                  AGE
VERSION
static.29.56.180.157.clients.your-server.de   Ready    control-plane,master   10m
v1.35.6
```

Complete Scripted Install

Here's the entire process as a single script. **Edit the variables at the top** for your environment.

`install-sno.sh`

```
#!/bin/bash
set -euo pipefail

# ============================================================
# EDIT THESE VARIABLES FOR YOUR ENVIRONMENT
# ============================================================
CLUSTER_NAME="ocp4"
BASE_DOMAIN="lapthorn.me"
OCP_VERSION="4.22"
SERVER_IP="157.180.56.29"
GATEWAY="157.180.56.1"
NIC_NAME="enp6s0"
MAC_ADDRESS="9c:6b:00:26:4e:7d"
MACHINE_NETWORK="157.180.56.0/26"
SSH_PUB_KEY="ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIF81+EYvBMyyg9dya8VpwmjiE/
UOL3gBxAPo7mrI4I92"
PULL_SECRET_FILE="pull-secret.json"
ISO_DISK="/dev/nvme0n1"          # Disk to write discovery ISO to
INSTALL_DISK="/dev/nvme1n1"      # Disk to install OpenShift to
# ============================================================

WORK_DIR="${CLUSTER_NAME}-install"
mkdir -p "$WORK_DIR"
cd "$WORK_DIR"

echo "=== Step 1: Create cluster-params.yml ==="
cat > cluster-params.yml <<EOF
openshift_version: "${OCP_VERSION}"
sno: true
base_dns_domain: ${BASE_DOMAIN}
pull_secret: ../${PULL_SECRET_FILE}
ssh_public_key: ${SSH_PUB_KEY}
api_vips: [${SERVER_IP}]
ingress_vips: [${SERVER_IP}]
machine_networks: ["${MACHINE_NETWORK}"]
network_type: OVNKubernetes
EOF

echo "=== Step 2: Create infraenv-params.yml ==="
cat > infraenv-params.yml <<EOF
cluster: ${CLUSTER_NAME}
static_network_config:
  - mac_interface_map:
      - logical_nic_name: ${NIC_NAME}
        mac_address: "${MAC_ADDRESS}"
    network_yaml: |
      interfaces:
        - name: ${NIC_NAME}
          type: ethernet
          state: up
          ipv4:
            enabled: true
            address:
              - ip: ${SERVER_IP}
                prefix-length: 32
            dhcp: false
          ipv6:
            enabled: false
      dns-resolver:
        config:
```

```
          server:
            - 185.12.64.1
            - 185.12.64.2
      routes:
        config:
          - destination: ${GATEWAY}/32
            next-hop-interface: ${NIC_NAME}
          - destination: 0.0.0.0/0
            next-hop-address: ${GATEWAY}
            next-hop-interface: ${NIC_NAME}
EOF

echo "=== Step 3: Create cluster and infraenv ==="
aicli create cluster "$CLUSTER_NAME" --pf cluster-params.yml
aicli create infraenv "$CLUSTER_NAME" --pf infraenv-params.yml

echo "=== Step 4: Download discovery ISO ==="
aicli download iso "$CLUSTER_NAME"

echo ""
echo "============================================================"
echo "  ISO downloaded: ${WORK_DIR}/${CLUSTER_NAME}.iso"
echo "============================================================"
echo ""
echo "Now run the following steps MANUALLY:"
echo ""
echo "1. Upload the ISO to the Hetzner server:"
echo "   scp ${CLUSTER_NAME}.iso root@${SERVER_IP}:/root/"
echo ""
echo "2. SSH to the rescue system and write the ISO:"
echo "   ssh root@${SERVER_IP}"
echo "   wipefs -af ${ISO_DISK}"
echo "   wipefs -af ${INSTALL_DISK}"
echo "   dd if=/root/${CLUSTER_NAME}.iso of=${ISO_DISK} bs=4M status=progress"
echo "   sync"
echo ""
echo "3. Create EFI boot entry and reboot:"
echo "   efibootmgr -c -d ${ISO_DISK} -p 2 -l \\\\EFI\\\\BOOT\\\\BOOTX64.EFI -L
'OpenShift Discovery'"
echo "   # Deactivate rescue in Hetzner Robot, then:"
echo "   reboot"
echo ""
echo "4. Wait for host discovery (5-10 minutes), then run:"
echo "   ./post-boot.sh"
echo ""
```

`post-boot.sh`

Run this after the discovery ISO boots and the host appears in `aicli list host`:

```
#!/bin/bash
set -euo pipefail

CLUSTER_NAME="ocp4"
INSTALL_DISK="/dev/nvme1n1"

echo "=== Checking host registration ==="
aicli list host
echo ""

# Get the hostname from the registered host
HOSTNAME=$(aicli list host -o json | python3 -c "
import sys, json
hosts = json.load(sys.stdin)
for h in hosts:
    if h.get('cluster') == '${CLUSTER_NAME}':
        print(h['host'])
        break
")

if [ -z "$HOSTNAME" ]; then
    echo "ERROR: No host found for cluster ${CLUSTER_NAME}"
    echo "Wait a few more minutes and try again."
    exit 1
fi

echo "Found host: $HOSTNAME"

echo "=== Setting installation disk to ${INSTALL_DISK} ==="
aicli update host "$HOSTNAME" -P installation_disk_path=${INSTALL_DISK}

echo "=== Verifying cluster readiness ==="
aicli info cluster "$CLUSTER_NAME" | grep -E 'status|ready'

echo ""
read -p "Cluster looks ready? Start install? (y/n) " -n 1 -r
echo ""

if [[ $REPLY =~ ^[Yy]$ ]]; then
    echo "=== Starting installation ==="
    aicli start cluster "$CLUSTER_NAME"

    echo ""
    echo "Installation started. Monitor with:"
    echo "  watch -n 30 'aicli info cluster ${CLUSTER_NAME} | grep -E \"status|
progress\"'"
    echo ""
    echo "After completion, download credentials with:"
    echo "  aicli download kubeconfig ${CLUSTER_NAME}"
    echo "  aicli download credentials ${CLUSTER_NAME}"
fi
```

`post-install.sh`

Run this after the install completes to set up EFI boot and download credentials:

```
#!/bin/bash
set -euo pipefail

CLUSTER_NAME="ocp4"
SERVER_IP="157.180.56.29"
INSTALL_DISK="/dev/nvme1n1"

echo "=== Downloading kubeconfig ==="
aicli download kubeconfig "$CLUSTER_NAME"
mkdir -p ~/sno-install
cp "${CLUSTER_NAME}/kubeconfig" ~/sno-install/kubeconfig.${CLUSTER_NAME}

echo "=== Downloading credentials ==="
aicli download credentials "$CLUSTER_NAME"

echo ""
echo "Kubeconfig saved to: ~/sno-install/kubeconfig.${CLUSTER_NAME}"
echo ""
echo "If the server is unreachable after the install reboot:"
echo "  1. Activate rescue in Hetzner Robot"
echo "  2. Hardware reset the server"
echo "  3. SSH to rescue: ssh root@${SERVER_IP}"
echo "  4. Create EFI boot entry:"
echo "     mount /dev/nvme1n1p2 /mnt"
echo "     efibootmgr -c -d ${INSTALL_DISK} -p 2 -l \\\\EFI\\\\redhat\\\\shimx64.efi -L
'RHEL'"
echo "     reboot"
echo ""

export KUBECONFIG=~/sno-install/kubeconfig.${CLUSTER_NAME}
echo "=== Verifying cluster ==="
oc get clusterversion
oc get nodes
echo ""
echo "=== Console URL ==="
oc whoami --show-console
```

Make scripts executable:

```
chmod +x install-sno.sh post-boot.sh post-install.sh
```

Troubleshooting

Host never appears in `aicli list host`

The discovery ISO couldn't reach the assisted installer API. Most common cause: **wrong static network config.**

1. Activate rescue in Hetzner Robot → hardware reset
2. SSH to rescue
3. Check your NIC name: `ip link` — Hetzner uses different NIC names per server model (e.g., `enp0s31f6`, `enp6s0`, `eno1`)
4. Check gateway: `ip route | grep default`

5. Ensure you have the /32 host route to the gateway in your `infraenv-params.yml`

Server unreachable after install reboot

The BIOS doesn't know which disk to boot from.

1. Activate rescue → reset
2. SSH to rescue
3. Check which disk has the OS:

```
lsblk
fdisk -l /dev/nvme1n1 | head -20
```

1. Mount EFI and create boot entry:

```
mount /dev/nvme1n1p2 /mnt
ls /mnt/EFI/
efibootmgr -c -d /dev/nvme1n1 -p 2 -l \\EFI\\redhat\\shimx64.efi -L "RHEL"
efibootmgr    # verify it's first in boot order
```

1. Deactivate rescue in Robot, then `reboot`

Install stuck at "Waiting for control plane"

SSH to the node and check:

```
ssh core@<SERVER_IP>
sudo crictl ps | grep -c Running    # Should be 50+ containers
journalctl -u kubelet --no-pager | tail -30
```

`aicli` authentication errors

```
# Re-export your offline token
export AI_OFFLINETOKEN='<token>'

# Or create ~/.aicli/config.yml:
echo "default:" > ~/.aicli/config.yml
echo "  token: <token>" >> ~/.aicli/config.yml
```

Disk conflict — both NVMe have the ISO

If you accidentally wrote the ISO to both disks:

```
# In rescue mode, wipe the second disk
wipefs -af /dev/nvme1n1
```

The live OS auto-mounts ISO partitions at `/run/media/iso`, blocking the installer from writing to that disk.

Quick Reference — aicli Commands

```
# Cluster lifecycle
aicli create cluster <name> --pf params.yml
aicli create infraenv <name> --pf params.yml
aicli download iso <name>
aicli list host
aicli update host <hostname> -P installation_disk_path=/dev/nvme1n1
aicli start cluster <name>
aicli info cluster <name>
aicli download kubeconfig <name>
aicli download credentials <name>

# Debugging
aicli info host <hostname>                    # Host details and validations
aicli info cluster <name> -f validations_info # Why isn't the cluster ready?
aicli list event --cluster <name>             # Installation events
aicli download installconfig <name>           # Generated install-config

# Cleanup
aicli delete cluster <name>
aicli delete infraenv <name>
```

Ansible Playbook — Fully Automated Install

This playbook automates the entire SNO installation on Hetzner. It runs from your workstation and handles: aicli cluster creation, ISO generation, uploading to the server, disk preparation, EFI boot setup, host discovery wait, install disk selection, install start, progress monitoring, and credential download.

> **Prerequisite:** The Hetzner server must be in **rescue mode** before running the playbook. Activate it in Hetzner Robot.

Directory Structure

```
sno-ansible/
├── inventory.yml
├── vars.yml
├── pull-secret.json
└── playbook.yml
```

inventory.yml

```
all:
  hosts:
    hetzner:
      ansible_host: 157.180.56.29
      ansible_user: root
      ansible_ssh_private_key_file: ~/.ssh/hetzner-sno
```

vars.yml

```
# Cluster
cluster_name: ocp4
base_domain: lapthorn.me
ocp_version: "4.22"
network_type: OVNKubernetes

# Server
server_ip: 157.180.56.29
gateway: 157.180.56.1
nic_name: enp6s0
mac_address: "9c:6b:00:26:4e:7d"
machine_network: "157.180.56.0/26"
ssh_pub_key: "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIF81+EYvBMyyg9dya8VpwmjiE/
UOL3gBxAPo7mrI4I92"
pull_secret_file: pull-secret.json

# Disks
iso_disk: /dev/nvme0n1
install_disk: /dev/nvme1n1

# DNS servers (Hetzner default)
dns_servers:
  - 185.12.64.1
  - 185.12.64.2

# Timeouts
host_discovery_timeout: 600    # seconds to wait for host to register
install_timeout: 3600          # seconds to wait for install to complete
```

`playbook.yml`

```
---
# SNO on Hetzner — end-to-end install with aicli
#
# Usage:
#   1. Activate rescue mode in Hetzner Robot
#   2. ansible-playbook -i inventory.yml playbook.yml
#
# The playbook pauses after writing the ISO to disk. You must:
#   - Deactivate rescue in Hetzner Robot (the playbook tells you when)
#   - Press Enter to continue

- name: "Phase 1: Create cluster and ISO (runs locally)"
  hosts: localhost
  connection: local
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Create work directory
      ansible.builtin.file:
        path: "./{{ cluster_name }}-install"
        state: directory
        mode: "0755"

    - name: Template cluster-params.yml
      ansible.builtin.copy:
        dest: "./{{ cluster_name }}-install/cluster-params.yml"
        content: |
          openshift_version: "{{ ocp_version }}"
          sno: true
          base_dns_domain: {{ base_domain }}
          pull_secret: ../{{ pull_secret_file }}
          ssh_public_key: {{ ssh_pub_key }}
          api_vips: [{{ server_ip }}]
          ingress_vips: [{{ server_ip }}]
          machine_networks: ["{{ machine_network }}"]
          network_type: {{ network_type }}

    - name: Template infraenv-params.yml
      ansible.builtin.copy:
        dest: "./{{ cluster_name }}-install/infraenv-params.yml"
        content: |
          cluster: {{ cluster_name }}
          static_network_config:
            - mac_interface_map:
                - logical_nic_name: {{ nic_name }}
                  mac_address: "{{ mac_address }}"
              network_yaml: |
                interfaces:
                  - name: {{ nic_name }}
                    type: ethernet
                    state: up
                    ipv4:
                      enabled: true
                      address:
                        - ip: {{ server_ip }}
                          prefix-length: 32
                      dhcp: false
                    ipv6:
```

```
                  enabled: false
            dns-resolver:
              config:
                server:
                  {% for dns in dns_servers %}- {{ dns }}
                  {% endfor %}
            routes:
              config:
                - destination: {{ gateway }}/32
                  next-hop-interface: {{ nic_name }}
                - destination: 0.0.0.0/0
                  next-hop-address: {{ gateway }}
                  next-hop-interface: {{ nic_name }}

- name: Check if cluster already exists
  ansible.builtin.command:
    cmd: "aicli info cluster {{ cluster_name }}"
  register: cluster_check
  failed_when: false
  changed_when: false

- name: Create cluster in assisted installer
  ansible.builtin.command:
    cmd: "aicli create cluster {{ cluster_name }} --pf cluster-params.yml"
    chdir: "./{{ cluster_name }}-install"
  when: cluster_check.rc != 0

- name: Check if infraenv already exists
  ansible.builtin.command:
    cmd: "aicli info infraenv {{ cluster_name }}"
  register: infraenv_check
  failed_when: false
  changed_when: false

- name: Create infraenv
  ansible.builtin.command:
    cmd: "aicli create infraenv {{ cluster_name }} --pf infraenv-params.yml"
    chdir: "./{{ cluster_name }}-install"
  when: infraenv_check.rc != 0

- name: Download discovery ISO
  ansible.builtin.command:
    cmd: "aicli download iso {{ cluster_name }}"
    chdir: "./{{ cluster_name }}-install"
    creates: "./{{ cluster_name }}-install/{{ cluster_name }}.iso"

- name: Verify ISO exists
  ansible.builtin.stat:
    path: "./{{ cluster_name }}-install/{{ cluster_name }}.iso"
  register: iso_file

- name: Fail if ISO not found
  ansible.builtin.fail:
    msg: "ISO file not found at ./{{ cluster_name }}-install/{{ cluster_name }}.iso"
  when: not iso_file.stat.exists

- name: Show ISO size
  ansible.builtin.debug:
    msg: "ISO size: {{ (iso_file.stat.size / 1048576) | round(1) }} MB"
```

```
- name: "Phase 2: Prepare server and write ISO (runs on Hetzner rescue)"
  hosts: hetzner
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Wait for rescue system to be reachable
      ansible.builtin.wait_for_connection:
        timeout: 120

    - name: Wipe ISO disk
      ansible.builtin.command:
        cmd: "wipefs -af {{ iso_disk }}"

    - name: Wipe install disk
      ansible.builtin.command:
        cmd: "wipefs -af {{ install_disk }}"

    - name: Upload discovery ISO to server
      ansible.builtin.copy:
        src: "./{{ cluster_name }}-install/{{ cluster_name }}.iso"
        dest: "/root/{{ cluster_name }}.iso"
        mode: "0644"

    - name: Write ISO to disk
      ansible.builtin.command:
        cmd: "dd if=/root/{{ cluster_name }}.iso of={{ iso_disk }} bs=4M"

    - name: Sync filesystem
      ansible.builtin.command:
        cmd: sync

    - name: Create EFI boot entry for discovery ISO
      ansible.builtin.command:
        cmd: >
          efibootmgr -c -d {{ iso_disk }} -p 2
          -l \EFI\BOOT\BOOTX64.EFI -L "OpenShift Discovery"

    - name: Show EFI boot order
      ansible.builtin.command:
        cmd: efibootmgr
      register: efi_output

    - name: Display EFI boot entries
      ansible.builtin.debug:
        var: efi_output.stdout_lines

    - name: "ACTION REQUIRED: Deactivate rescue in Hetzner Robot"
      ansible.builtin.pause:
        prompt: |
          ============================================================
          Go to Hetzner Robot → Server → Rescue → DEACTIVATE rescue
          Then press ENTER to reboot the server
          ============================================================

    - name: Reboot server (soft reboot — preserves EFI)
      ansible.builtin.command:
        cmd: reboot
      async: 1
      poll: 0
```

```
      ignore_errors: true


- name: "Phase 3: Wait for discovery, set install disk, start install (runs locally)"
  hosts: localhost
  connection: local
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Wait for server to go down
      ansible.builtin.pause:
        seconds: 60
        prompt: "Waiting 60s for server to reboot into discovery ISO..."

    - name: Wait for host to register with assisted installer
      ansible.builtin.command:
        cmd: "aicli list host -o json"
      register: host_list
      until: >

(host_list.stdout | from_json | selectattr('cluster', 'equalto', cluster_name) | list |
length) > 0
      retries: "{{ (host_discovery_timeout / 30) | int }}"
      delay: 30
      changed_when: false

    - name: Extract hostname
      ansible.builtin.set_fact:
        discovered_host: >-
          {{ (host_list.stdout | from_json |
              selectattr('cluster', 'equalto', cluster_name) |
              first).host }}

    - name: Display discovered host
      ansible.builtin.debug:
        msg: "Host registered: {{ discovered_host }}"

    - name: Wait for host status to become 'known'
      ansible.builtin.command:
        cmd: "aicli info host {{ discovered_host }} -o json"
      register: host_info
      until: >
        (host_info.stdout | from_json).status == 'known'
      retries: 20
      delay: 30
      changed_when: false

    - name: Set installation disk
      ansible.builtin.command:
        cmd: >
          aicli update host {{ discovered_host }}
          -P installation_disk_path={{ install_disk }}

    - name: Verify cluster is ready
      ansible.builtin.command:
        cmd: "aicli info cluster {{ cluster_name }} -o json"
      register: cluster_info
      until: >
        (cluster_info.stdout | from_json).status == 'ready'
```

```
      retries: 10
      delay: 15
      changed_when: false

    - name: Start installation
      ansible.builtin.command:
        cmd: "aicli start cluster {{ cluster_name }}"

    - name: Wait for installation to complete
      ansible.builtin.command:
        cmd: "aicli info cluster {{ cluster_name }} -o json"
      register: install_status
      until: >
        (install_status.stdout | from_json).status == 'installed'
      retries: "{{ (install_timeout / 60) | int }}"
      delay: 60
      changed_when: false

    - name: Installation complete
      ansible.builtin.debug:
        msg: "OpenShift SNO installation completed successfully!"


- name: "Phase 4: Download credentials and verify (runs locally)"
  hosts: localhost
  connection: local
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Create credentials directory
      ansible.builtin.file:
        path: "~/sno-install"
        state: directory
        mode: "0700"

    - name: Download kubeconfig
      ansible.builtin.command:
        cmd: "aicli download kubeconfig {{ cluster_name }}"
        chdir: "./{{ cluster_name }}-install"

    - name: Copy kubeconfig
      ansible.builtin.copy:
        src: "./{{ cluster_name }}-install/{{ cluster_name }}/kubeconfig"
        dest: "~/sno-install/kubeconfig.{{ cluster_name }}"
        mode: "0600"

    - name: Download credentials
      ansible.builtin.command:
        cmd: "aicli download credentials {{ cluster_name }}"
        chdir: "./{{ cluster_name }}-install"

    - name: Wait for API to become reachable
      ansible.builtin.command:
        cmd: "oc get clusterversion"
      environment:
        KUBECONFIG: "~/sno-install/kubeconfig.{{ cluster_name }}"
      register: cv_output
      until: cv_output.rc == 0
      retries: 30
```

```
      delay: 30
      changed_when: false

    - name: Get cluster version
      ansible.builtin.command:
        cmd: "oc get clusterversion"
      environment:
        KUBECONFIG: "~/sno-install/kubeconfig.{{ cluster_name }}"
      register: cv
      changed_when: false

    - name: Get nodes
      ansible.builtin.command:
        cmd: "oc get nodes"
      environment:
        KUBECONFIG: "~/sno-install/kubeconfig.{{ cluster_name }}"
      register: nodes
      changed_when: false

    - name: Get console URL
      ansible.builtin.command:
        cmd: "oc whoami --show-console"
      environment:
        KUBECONFIG: "~/sno-install/kubeconfig.{{ cluster_name }}"
      register: console_url
      changed_when: false

    - name: Installation summary
      ansible.builtin.debug:
        msg: |
          ============================================================
          SNO Installation Complete!
          ============================================================

          Cluster Version:
          {{ cv.stdout }}

          Nodes:
          {{ nodes.stdout }}

          Console: {{ console_url.stdout }}
          Kubeconfig: ~/sno-install/kubeconfig.{{ cluster_name }}
          ============================================================
```

Running the Playbook

```
# Install ansible if needed
pip3 install ansible

# Edit vars.yml with your server details
vi sno-ansible/vars.yml

# Ensure rescue mode is active in Hetzner Robot, then:
cd sno-ansible
ansible-playbook -i inventory.yml playbook.yml
```

The playbook pauses once to let you deactivate rescue mode in Hetzner Robot. Everything else is automated — including waiting for host discovery and monitoring installation progress.

Total runtime: ~45-60 minutes (mostly waiting for the OpenShift install to complete).

Hetzner-Specific Gotchas Summary

Gotcha	Impact	Fix
/32 IP addressing	No route to gateway, no network	Add explicit host route: `destination: <gw>/32`
NIC names vary per model	Static config uses wrong interface	Check `ip link` in rescue before creating infraenv
Dual NVMe — ISO on both	Installer can't write to second disk	Only `dd` ISO to ONE disk, wipe the other
EFI boot entry missing	Server unreachable after reboot	Create entry with `efibootmgr` before rebooting
Hardware reset wipes EFI	Boot entry lost after Hetzner Robot reset	Always use soft `reboot`, rescue → soft reset only
Same disk install target	Installer tries to overwrite its own ISO	Set `installation_disk_path` to the OTHER disk